

SETEMBRO | 2026



Teste Código Grupo 1

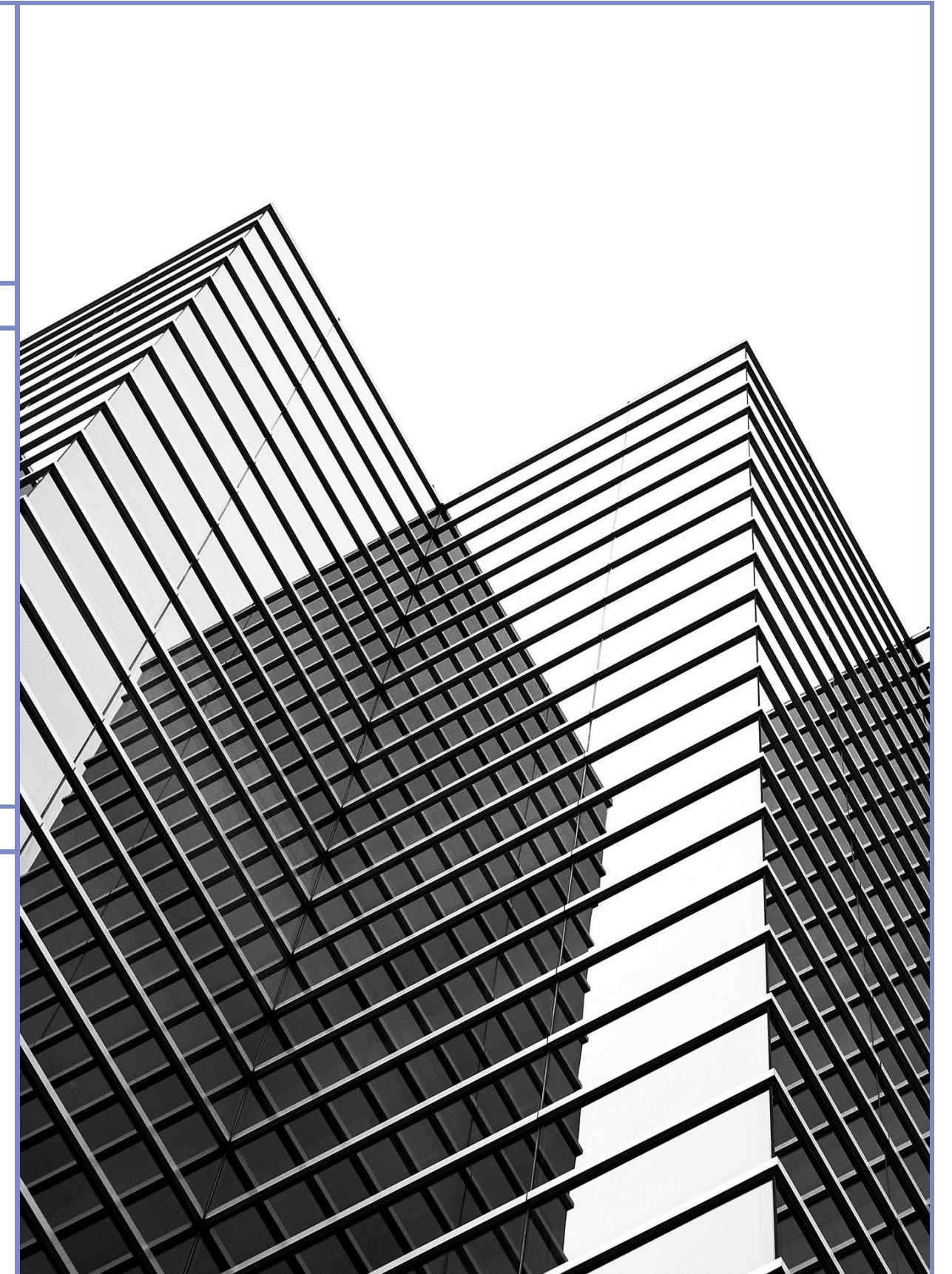
Alunos: Mateus Silvério e Rafael
Machado.

Linguagens e Paradigamas de
Programação

Código em python executado no terminal do VS Code e num compilador online.

Inicialmente, o sistema foi executado corretamente, sem apresentar erros de sintaxe ou semântica. Após uma análise mais detalhada, identificamos um erro durante a execução, que será descrito e analisado.

Quando induzimos a um erro, por exemplo: cadastrar aluno sem nome, o código não parou de executar e seguiu devido tratamento de erros e exceções.



Erro Identificado

Foi identificado um problema na busca de alunos quando existiam dois ou mais cadastros com o mesmo nome. A lógica anterior interrompia a procura assim que encontrava a primeira ocorrência, fazendo com que os demais alunos com o mesmo nome não fossem considerados. Isso poderia levar à seleção ou alteração do aluno incorreto e dificultava a consulta de cadastros duplicados.

```
312 # =====
313 # 7 - CONSULTAR ALUNO
314 # =====
315
316 ✓ def consultar_aluno():
317
318     print("\n=====")
319     print("CONSULTAR ALUNO")
320     print("=====")
321
322 ✓     if len(alunos) == 0:
323         print("Nenhum aluno cadastrado.")
324         return
325
326     nome = input("Digite o nome do aluno: ").strip()
327
328     aluno = buscar_aluno(nome)
329
330 ✓     if aluno is None:
331
332         print("Erro: aluno não encontrado.")
333
334         return
335
336     nota = aluno["nota"]
337
338     print("\nAluno encontrado:")
339     print(f"Nome: {aluno['nome']}")
340     print(f"Nota: {nota:.2f}")
341     print(f"Situação: {situacao(nota)}")
342     print(f"Desempenho: {desempenho(nota)}")
343
```

Erro Identificado

```
39
40 def buscar_aluno(nome):
41     for aluno in alunos:
42         if aluno["nome"].lower() == nome.lower():
43             return aluno
44
45     return None
46
```

Erro Identificado Corrigido

A busca foi modificada para percorrer toda a lista de alunos e armazenar todas as ocorrências encontradas em uma nova lista de resultados. Dessa forma, quando existem alunos com o mesmo nome, o sistema apresenta todas as opções e permite que o usuário escolha exatamente qual cadastro deseja consultar, alterar ou excluir. A correção tornou o processo mais seguro, consistente e adequado para situações com nomes repetidos.

```
def buscar_aluno(nome):
    encontrados = []

    for aluno in alunos:
        if aluno["nome"].lower() == nome.lower():
            encontrados.append(aluno)

    return encontrados

def selecionar_aluno():
    if len(alunos) == 0:
        print("Nenhum aluno cadastrado.")
        return None

    nome = input("Digite o nome do aluno: ").strip()

    encontrados = buscar_aluno(nome)

    if len(encontrados) == 0:
        print("Erro: aluno não encontrado.")
        return None

    if len(encontrados) == 1:
        return encontrados[0]

    print("\nForam encontrados vários alunos:")

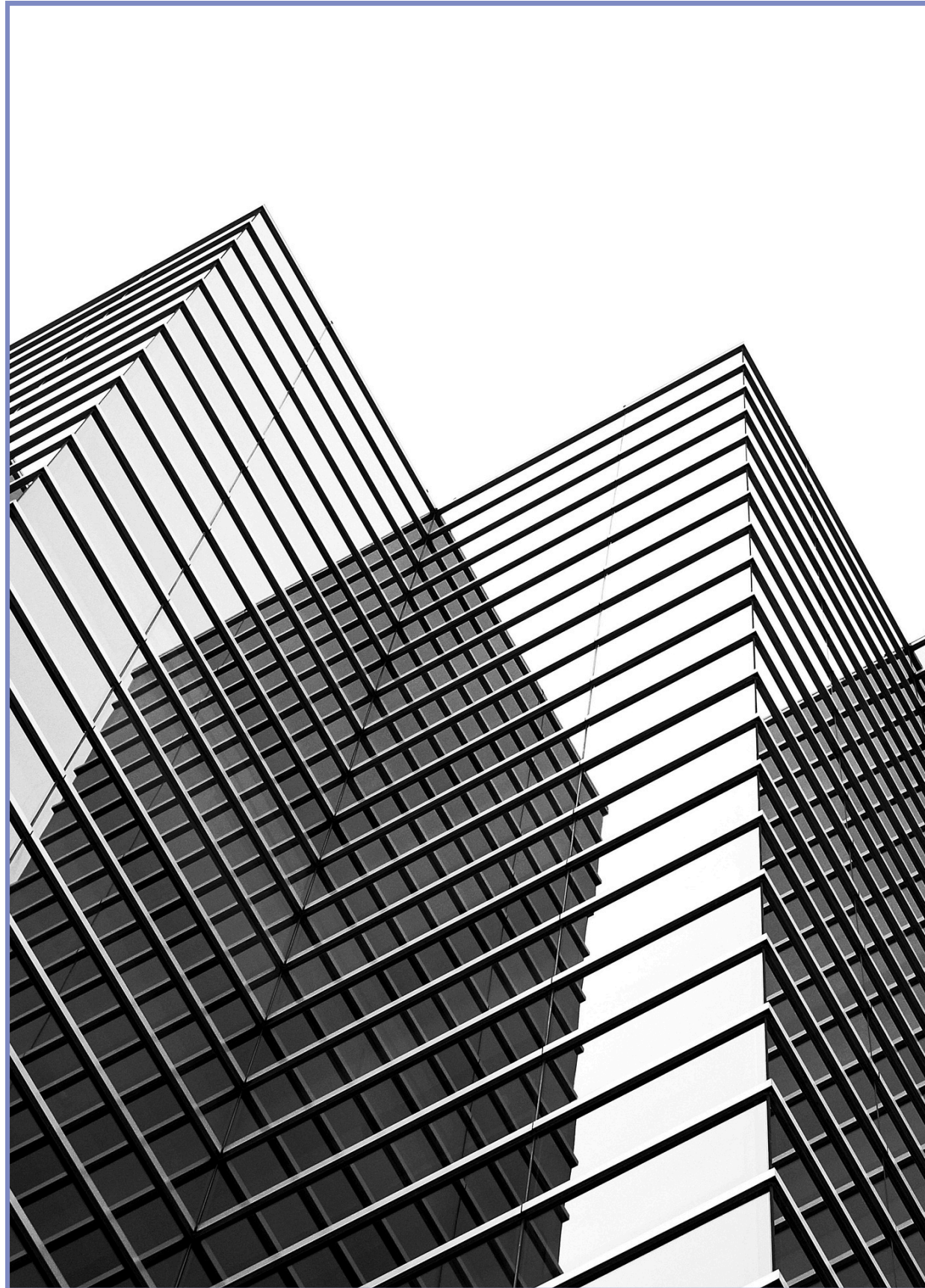
    for i, aluno in enumerate(encontrados):
        print(f"{i + 1} - {aluno['nome']} - Nota: {aluno['nota']:.2f}")

    while True:
        try:
            escolha = int(input("\nEscolha o número do aluno: "))

            if escolha < 1 or escolha > len(encontrados):
                print("Erro: escolha uma opção válida.")
                continue

            return encontrados[escolha - 1]

        except ValueError:
            print("Erro: digite um número inteiro.")
```



Python : linguagem multiparadigma

Paradigma Procedural (ou Imperativo)

Foca em passos sequenciais, comandos e atribuições de variáveis que instruem o computador sobre como fazer algo passo a passo. Ex: for, while, Declaração de variáveis simples;

Paradigma Funcional

Foca no uso de funções puras (que não alteram estados externos) e na imutabilidade de dados, tratando funções como cidadãos de primeira classe. Ex: Funções anônimas (lambda): `dobro = lambda x: x * 2`; Funções de alta ordem integradas como map, filter e reduce;

Paradigma Orientado a Objetos (POO)

Foca na criação de classes e objetos, modelando entidades do mundo real com atributos (dados) e métodos (comportamentos). Ex: Definição de classes, Uso de herança e encapsulamento;

Pontos de Melhoria

Algumas funcionalidades possuem trechos de lógica semelhantes que poderiam ser reorganizados para evitar repetição de código.

Algumas partes do código poderiam receber comentários para explicar e facilitar a compreensão das funções.

A organização e identificação de algumas funções poderia ser aprimorada para facilitar a manutenção do código.



Novas funcionalidades

Alteração de nota: permite localizar um aluno e atualizar somente sua nota, mantendo os demais dados do cadastro.

Edição de aluno: possibilita modificar o nome e/ou a nota de um cadastro existente, permitindo manter os valores atuais quando o usuário não deseja alterá-los.

Exclusão de aluno: permite selecionar um cadastro e removê-lo da lista após uma confirmação, evitando exclusões acidentais.

Prova de recuperação: adiciona uma pequena avaliação ao sistema. O aluno responde duas questões e, caso obtenha os dois acertos, sua nota é atualizada para 10.



**Obrigado pela
Atenção**

